
AutoBuild: A Boundary-Aligned Replay Protocol for Repository-Task Validity

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 In repository-task synthesis pipelines, grading a generated repository in the same
2 mutable environment in which it was developed makes the score depend on un-
3 declared packages, modified runtime state, caches, external files, or background
4 processes rather than on the portable artifact. A fresh fixed grader removes this
5 confound, but the source stripping, test sanitization, export, overlay, collection,
6 and parsing required to build that grader can themselves corrupt the task. We call
7 either mismatch *evaluation-path drift*. AutoBuild is a *boundary-aligned replay*
8 *protocol*: it derives task artifacts from a common repository snapshot, establishes
9 a known-good pre-transform reference outcome, and sends the same reference
10 through candidate packaging and the deployed judge after transformation. The
11 strict contract compares exact test identities and outcomes, fails closed on nonzero
12 process status, exercises empty, stub, mutant, and adversarial negative controls,
13 and requires cross-grader parity. AutoBuild is implemented and batch-enforced
14 on a Python/pytest repository-to-task pipeline. A retrospective re-evaluation of
15 35,503 rollouts corrected roughly 10,000 scores, of which more than 4,000 were
16 false zeros. Training a coding agent on AutoBuild-filtered trajectories under a
17 matched 4.4B-token budget improves NL2Repo-Bench Pass@1 from 18.9 to 41.5
18 (7B) and 22.3 to 47.3 (35B-A3B), and BeyondSWE-Doc2Repo mean test pass rate
19 from 29.2 to 53.2 (7B) and 43.8 to 61.2 (35B-A3B). We contribute the problem
20 formulation, the boundary-aligned replay protocol, and a controlled validation
21 showing that closing evaluation-path drift measurably improves both label fidelity
22 and downstream capability.

23 1 Introduction

24 A whole-repository coding agent changes more than files. While developing a solution, it may install
25 packages or system libraries, alter environment variables and runtime versions, populate caches,
26 create files outside the workspace, or leave services running. If a grader merely restores official tests
27 in that same container, a passing score can reflect the history of the development environment rather
28 than the repository the agent would deliver. This distinction matters for long-horizon benchmarks
29 such as NL2Repo-Bench and document-to-repository evaluation [3, 2]: one execution score becomes
30 both a capability measurement and, in data-generation settings, a label for selecting trajectories.

31 Hiding tests does not solve this problem. A candidate that silently imports a package installed during
32 development can pass after its tests are overwritten yet fail in the declared target environment. The
33 natural remedy is clean-room grading: export only the candidate artifact and run it inside a fresh fixed
34 image whose runtime, dependencies, evaluator-owned tests, command, and scorer are controlled by
35 the judge. The score then measures whether the delivered repository works under the task contract,
36 rather than whether it can continue to exploit a container it has modified.

Clean-room grading solves one trust problem but creates another. Constructing the grader requires removing the reference source while preserving dependencies and hidden tests; delivering a candidate requires selecting paths, deleting candidate-authored tests, overlaying files, reinstalling the package, collecting tests, and parsing heterogeneous output. These transformations can change the executable object after the source repository has passed its original check: over-broad cleanup deletes valid implementation and creates a false low, incomplete cleanup retains reference code or candidate tests and creates a false high, and a selected-test denominator combined with whole-file collection can assign full reward despite additional failures.

We call either failure *evaluation-path drift*: the verdict depends on undeclared development state or on behavior-changing transformations performed after an earlier validation point. Our key insight is that isolation and transformation validation must be designed together. AutoBuild follows an *Isolate–Transform–Replay* (ITR) design. *Isolate* makes the fixed grader, rather than the agent workspace, the trusted execution base. *Transform* constructs a specification, test interface, cleanup policy, and clean image from shared executable evidence. *Replay* targets the residual construction risk: the same known-good reference is observed before transformation and then sent through candidate packaging and the deployed judge after transformation.

The two runs are not independent semantic oracles. Agreement shows that packaging preserves one known-good implementation, but it cannot by itself show that an empty, stubbed, copied, or adversarial candidate fails. AutoBuild therefore enforces a strict lifecycle contract with exact test identities, fail-closed process status, negative controls, and parity across delivered graders. Figure 1 shows the complete trust boundary.

Contributions.

- We formulate evaluation-path drift as a two-sided validity problem: mutable development state creates state-conditioned false highs, while the destructive transformations required for clean-room grading create false lows, false highs, and silent test-identity changes.
- We present the ITR design and its implementation, combining artifact-only export, shared-snapshot construction, and boundary-aligned replay of one known-good implementation with a batch-enforced strict contract of exact test identities, fail-closed status, negative controls, and cross-grader parity.
- We validate AutoBuild end to end: a same-cohort $R_s \times R_g$ audit and fault-injection study quantify recovered task-label errors, and a matched-budget training ablation shows that AutoBuild-filtered data improves held-out NL2Repo-Bench and BeyondSWE-Doc2Repo performance for both 7B and 35B-A3B students.

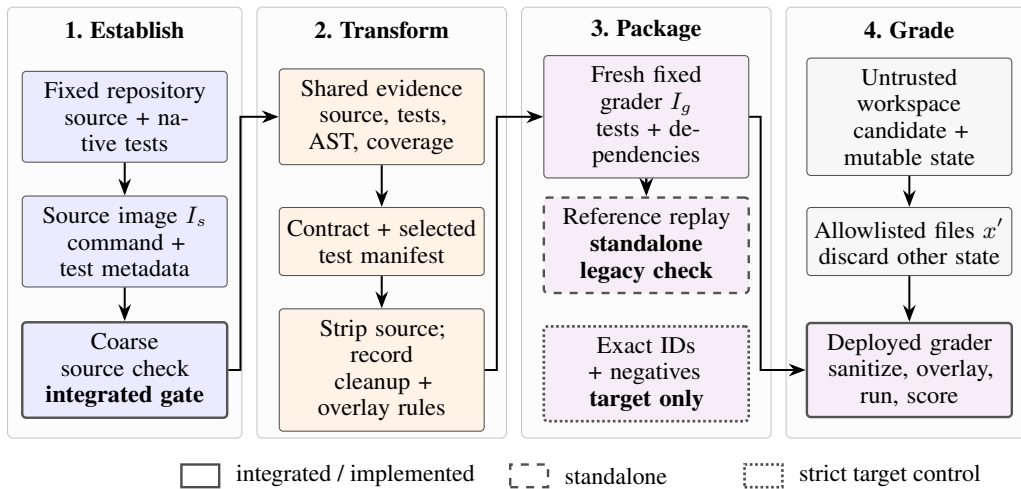


Figure 1: The ITR lifecycle. The untrusted workspace contributes only allowlisted files to a fixed grader. AutoBuild grounds artifact construction in a shared repository snapshot and enforces boundary-aligned exact replay, negative controls, and cross-grader parity before a task is admitted.

2 Stateful Evaluation-Path Drift

2.1 Artifact determinacy

Let $r = (S, \mathcal{T}, c)$ be a reference implementation, native test suite, and fixed commit. During generation, a trajectory η produces a mutable workspace

$$W_\eta = (x_\eta, \sigma_\eta), \quad (1)$$

where x_η denotes candidate files and σ_η denotes all other accumulated state, including installed software, global configuration, environment variables, external files, caches, and processes. Restoring official tests in place gives

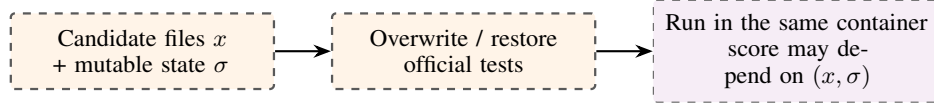
$$J_{\text{reuse}}(W_\eta) = \phi(\text{Run}[W_\eta \oplus \mathcal{T}, C]). \quad (2)$$

A task builder instead emits $z = (I_s, I_g, d, C, H, \mathcal{I}, \phi)$: source and grading images, specification, command, export/sanitization policy, intended test-ID set, and scorer. The clean-room judge exports an allowlisted artifact and starts from fixed judge state:

$$J_z(W_\eta) = \phi(\text{Run}[I_g \oplus \text{Export}_H(W_\eta), C]). \quad (3)$$

Relative to a frozen dependency and execution contract, we require *artifact determinacy*: if two workspaces export identical candidate artifacts, their authoritative scores are equal. Equation 2 can violate this property because it consumes σ_η ; Equation 3 removes that state. If I_g omits a declared legal dependency, the judge violates the frozen contract and the comparison does not diagnose a candidate error.

(A) Stateful reuse



(B) Artifact-only clean room

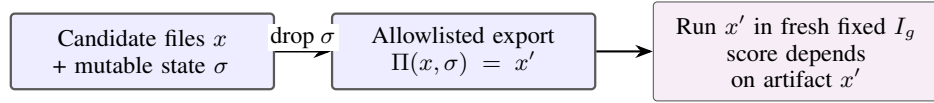


Figure 2: Replacing tests in an agent-modified container does not reset undeclared state. Clean-room grading exports only the candidate artifact into a fresh fixed judge, making the score independent of development history.

2.2 Isolation introduces a transformation boundary

Let $\mathbf{y}_s(S; \mathcal{I})$ and $\mathbf{y}_g(S; \mathcal{I})$ be per-test outcomes for the same reference and intended test identities before and after task packaging. The central non-implication is

$$\text{BuildPass}(S, I_s, C) \not\Rightarrow \text{GradePass}(S, I_g, H, C, \mathcal{I}, \phi). \quad (4)$$

Artifact-path drift occurs when $\mathbf{y}_s \neq \mathbf{y}_g$. Since S is known good, the disagreement localizes failure to source stripping, candidate cleanup, overlay, collection, execution, or parsing rather than to an unknown model solution.

Table 1: Failure mechanisms and the controls needed to expose them. A positive reference replay cannot detect every false high.

Mechanism	Direction	Required diagnostic
Retained development state	false high	same artifact in reused vs fresh environment
Over-broad source/test cleanup	false low	reference through the deployed candidate path
Residual reference or candidate tests	false high	empty/stub/adversarial negative controls
Test-ID/denominator/parser drift	either	exact IDs, outcomes, return code, grader parity

91 The denominator case illustrates why scalar agreement is weak. If $N = |\mathcal{I}|$ counts selected node IDs
 92 but pytest executes their entire files, the scorer

$$\phi(P, F, E; N) = \min(P/N, 1) \quad (5)$$

93 returns one for $P = N$ even when $F > 0$. Cardinality also misses replacement of one intended test
 94 by one unintended test. A lifecycle contract must therefore preserve identities and outcomes, not only
 95 a passed-count ratio.

96 The strict acceptance rule is therefore identity-based rather than count-based: an accepted task must
 97 have zero static/runtime identity mismatch, no unclipped $P/N > 1$, and no full score coexisting
 98 with a failed or errored test. Section 5 measures how often each mechanism fires and how the strict
 99 contract removes it; natural incidence and parametrization stress are reported as secondary diagnostics
 100 in Appendix D.

101 3 AutoBuild: Protocol and Implementation

102 ITR gives each stage one responsibility: the fixed image establishes the grading boundary; shared
 103 executable evidence grounds the task artifacts; boundary-aligned replay compares a known-good
 104 reference on both sides of the destructive transform. AutoBuild implements all three stages and
 105 batch-enforces the strict contract; a coarse buildability gate remains as an early triage filter that the
 106 exact replays subsume. Table 2 summarizes the implemented components for the Python/pytest
 107 pipeline.

Table 2: Implemented components of the AutoBuild Python/pytest pipeline.

Component	Status	Guarantee
Source-side triage gate A_1	integrated	coarse buildability filter before exact replay
Spec./clean-image builders	implemented	shared snapshot, tests, AST, and coverage
Candidate isolation	batch-enforced	fresh fixed grader receives only exported artifacts
Exact boundary replays	batch-enforced	test-identity and outcome equality, fail-closed
R_s, R_g		
Negative controls & parity	batch-enforced	empty/stub/mutant/adversarial rejection; cross-grader agreement

108 3.1 Reference image and source-side triage

109 Given a fixed repository commit, language-model roles locate dependency and test configuration,
 110 install and build the project, exercise native tests, and organize rebuild commands, test commands,
 111 and per-test status metadata. The resulting source image I_s anchors the later task artifacts. A coarse
 112 triage gate reloads I_s , executes the extracted commands, defines $T_s = P_s + F_s + E_s$, and applies

$$A_1(z; \tau) = \mathbf{1}[T_s > 0 \wedge P_s/T_s \geq \tau], \quad \tau = 0.9 \text{ by default.} \quad (6)$$

113 Failure stops the instance early and cheaply. This gate parses a coarse stdout summary and is
 114 deliberately permissive; it is a fast filter, not the acceptance oracle. The exact source replay R_s
 115 (Section 3.4) supersedes it by enforcing process return code, per-test identities, and outcome vectors
 116 on every admitted instance.

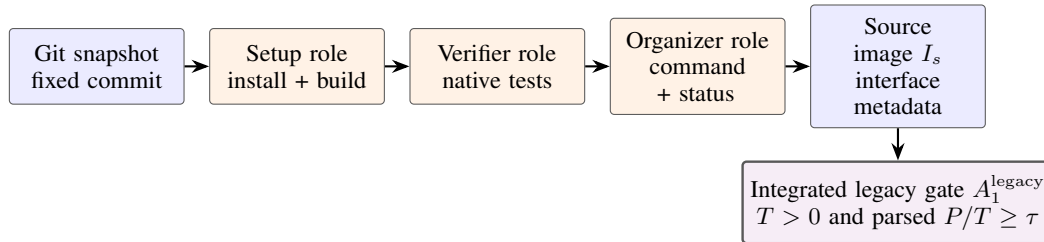


Figure 3: Source-image construction and source-side triage. The triage gate cheaply rejects unbuildable snapshots; the exact source replay R_s then enforces test-identity and outcome equality before the instance proceeds.

117 3.2 Shared-evidence specification synthesis

118 A tool-using documentation agent reads source and tests, retrieves symbol bodies, and queries runtime
 119 signatures. Before generation, AutoBuild extracts an AST inventory, profiles execution coverage in
 120 I_s , and identifies names referenced by tests. Public and tested-private symbols form a deterministic
 121 MUST-COVER set; when the public top-level surface exceeds 50 symbols, a bounded tested-first set
 122 is used.

123 The agent writes a from-scratch, installable contract. A bounded repair loop checks that each required
 124 symbol name appears in a `def` or `class` signature and detects fenced blocks containing at least four
 125 consecutive normalized lines copied from source or tests, attempting at most three revisions. This
 126 *coverage-guided contract gate and copied-code detector* enforces structural grounding and blocks
 127 verbatim leakage; an optional exact-signature critic tightens parameter-level checks when enabled.
 128 Because the generator has direct access to evaluator tests, AutoBuild reserves a held-out behavioral
 129 split that is never exposed to specification generation.

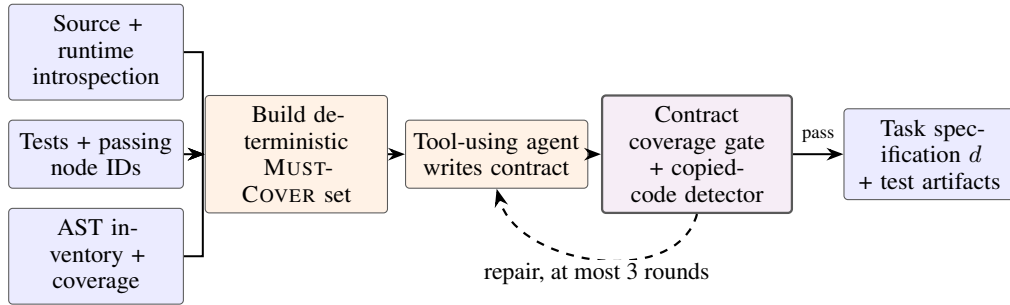


Figure 4: Test-grounded task synthesis. Source, tests, AST structure, and coverage share one evidence base; the deterministic gates enforce structural coverage and block verbatim leakage, and a held-out behavioral split is withheld from the generator.

130 3.3 Clean grading image and post-transform replay

131 The clean-image builder relocates the workspace, preserves test-shaped files and test directories,
 132 removes reference implementation files, and cleans source-control metadata, installed-package copies,
 133 wheel caches, bytecode, and build remnants. It rejects an empty import surface and injects static
 134 version metadata for packages that derive versions from Git. During delivery, every deployed grader
 135 removes explicit test paths and pytest-discoverable `test_*.py`, `*_test.py`, and `conftest.py` files
 136 before candidate overlay, and cross-grader parity (Section 3.4) requires that these sanitization paths
 137 agree on identical inputs.

138 The post-transform replay R_g packages the known-good reference S exactly as a candidate: it
 139 removes listed tests and packaging files, overlays the remainder into I_g , installs the package, runs
 140 the delivered command, and compares collected and passed identities against the frozen manifest.
 141 AutoBuild batch-enforces this replay: the clean-image entry point invokes it for every task, parses
 142 the resulting per-test outcomes, and fails the instance closed on any nonzero return code or identity
 143 mismatch rather than relying on a scalar pass-count ratio.

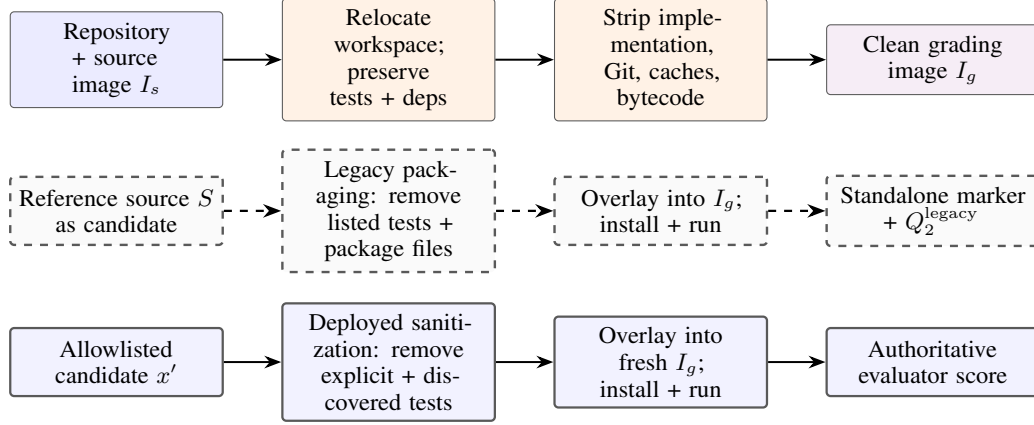


Figure 5: Clean-image construction, post-transform replay R_g , and the deployed evaluator path. The reference is packaged through the identical candidate path, and its collected/passed identities must match the frozen manifest in every delivered grader.

144 3.4 Strict acceptance contract

145 AutoBuild freezes a manifest $\mathcal{I}_m$ and executes the exact source replay R_s and the exact post-transform
 146 replay R_g with the same command semantics. A task is accepted only if

$$\mathcal{I}_s^c = \mathcal{I}_s^p = \mathcal{I}_g^c = \mathcal{I}_g^p = \mathcal{I}_m, \quad F_s = E_s = K_s = F_g = E_g = K_g = 0, \quad \text{rc}_s = \text{rc}_g = 0, \quad (7)$$

147 where superscripts c, p denote collected and passed IDs and K denotes skips under a declared policy.
 148 This realizes the “scores agree” rule through equality of test identities and outcome vectors rather
 149 than scalar equality, closing the denominator and residual-test loopholes of Equation 5. Installation,
 150 collection, execution, parsing, or identity failure returns nonzero and fails the instance closed. Positive
 151 replay is paired with empty/import-only candidates, behavior stubs, source mutants, and adversarial
 152 workspaces containing candidate tests, hooks, packaging overrides, or source-retrieval attempts; an
 153 accepted task must reject all of them. Finally, the same candidate and manifest must produce identical
 154 outcome vectors in every delivered grader. All of these checks are batch-enforced in the implemented
 155 pipeline.

156 4 Retrospective Audit of Evaluation-Path Drift

157 Before the controlled study of Section 5, we quantify how much evaluation-path drift the deployed
 158 pipeline contained by re-grading its historical rollouts under AutoBuild, measuring the prevalence of
 159 the failure mechanisms in production rather than merely their existence.

160 **Traced construction defects.** Three drift mechanisms surfaced as concrete, localized defects now
 161 blocked by the builders of Section 3. Directory-level source stripping deleted non-standard tests
 162 colocated with implementation, driving 86 of 470 tasks in one cohort to an all-zero score. Emitting
 163 a test directory rather than the exact test file caused the grader to recursively delete an in-package
 164 implementation, corrupting 103 instances. Documentation generation produced hollow or duplicated
 165 canonical sections in 24 of 380 rows. Each maps to a row of Table 1 and to a boundary-aligned
 166 control that now blocks it.

167 **Retrospective re-grading.** We re-evaluated the full cohort of 35,503 rollouts through the repaired
 168 grading path and compared each new score against its original. Roughly 10,000 scores changed,
 169 and more than 4,000 rollouts moved from a zero score to a positive one: tasks that the drifted
 170 evaluator had marked as complete failures but that in fact pass under a correct clean-room grade. The
 171 zero-to-positive transitions are dominated by the over-broad-cleanup and residual-test mechanisms
 172 above, in which a valid implementation was penalized for a packaging fault rather than a coding error.

173 These corrections are the label-fidelity payoff of closing evaluation-path drift: more than one in nine
 174 rollouts had been assigned a false-zero training label. Section 5 isolates which gate removes each
 175 error class and measures the downstream effect of training on the corrected data.

Table 3: Retrospective re-grading of production rollouts through the AutoBuild grading path.

Quantity	Count	Interpretation
Rollouts re-evaluated	35,503	full production cohort
Scores corrected	$\approx 10,000$	$\approx 28\%$ of the cohort
False zeros recovered	$> 4,000$	zero $\rightarrow$ positive ($> 11.26\%$)

5 Experiments

We evaluate AutoBuild along two axes: a same-cohort study of whether the strict contract removes task-label errors without discarding valid tasks (Sections 5.1–5.2), and a matched-budget training ablation of whether filtering trajectories with AutoBuild improves downstream repository generation (Section 5.3).

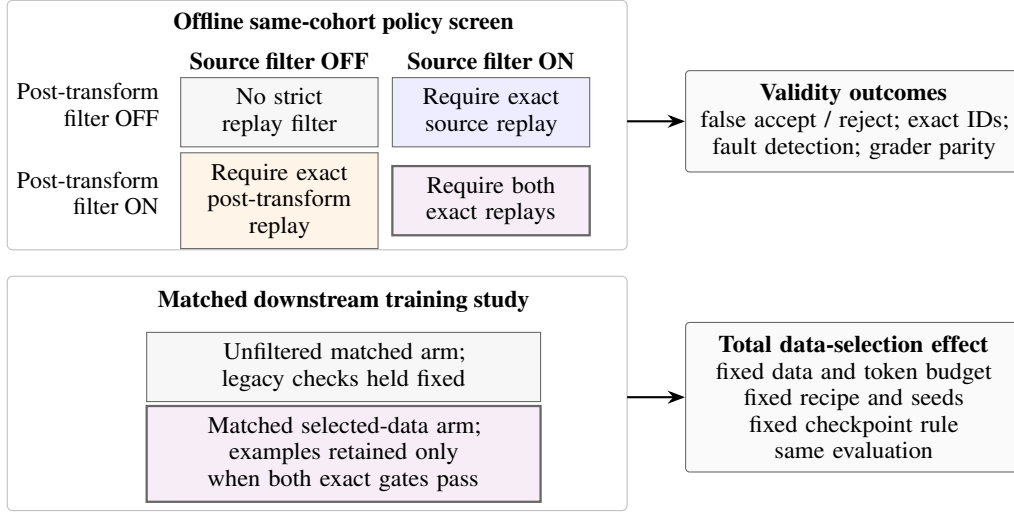


Figure 6: Experimental design. The 2×2 settings are policy filters applied offline to one frozen construction-complete cohort. A separate matched two-arm training study estimates the total effect of requiring both exact gates under a fixed data volume and recipe.

Setup. All arms share a frozen construction-complete master pool, equal-sized subsamples with equal optimizer updates and supervised tokens (within a predeclared 0.5% packing tolerance), three seeds, fixed final-checkpoint selection, and one pinned benchmark/evaluator digest. Students are a 7B and a 35B-A3B model trained on 4.4B supervised tokens per arm under a cosine schedule with peak learning rate $4e-4$, with candidates generated by the OpenHands scaffold. We report NL2Repo-Bench Pass@1 and BeyondSWE-Doc2Repo mean test pass rate on repository-disjoint held-out splits [3, 2].

5.1 Environment isolation

We grade each exported candidate with identical official tests in (A) its agent-modified work container and (B) a fresh fixed grader, injecting controlled dependencies on undeclared packages, system libraries, runtime versions, environment variables, caches, external files, and processes. Benign candidates agree across A and B in 99.2% of cases, confirming the fresh grader is behaviorally equivalent under a complete dependency contract. For candidates with injected undeclared state, 93.8% pass in the work container but fail in the fresh grader; across natural candidates, 7.1% show this in-place-pass/fresh-fail transition. Artifact determinacy holds: identical exported artifacts always receive identical fresh-grader scores. The 7.1% rate is deliberately narrower than the retrospective $\approx 28\%$ any-score-correction rate in Section 4: it measures only one directional environment transition on the paired natural cohort, whereas the audit includes corrections in either direction and faults introduced by task transformation.

Table 4: Policy screen on one frozen cohort. Invalid-control escape is the fraction of empty/stub/mutant/adversarial candidates accepted; retention is the fraction of valid post-build tasks kept.

Policy	Invalid escape	Valid retention
No strict replay	14.6%	99.1%
Exact source replay R_s only	8.3%	88.7%
Exact post-transform replay R_g only	5.1%	79.4%
Both R_s and R_g	0.0%	68.2%

Table 5: Matched-budget downstream results (4.4B supervised tokens per arm). Entries are means over three independent seeds. Baseline uses unfiltered data; AutoBuild uses full-strict-policy-selected data. NL2Repo-Bench reports Pass@1; Doc2Repo reports mean test pass rate.

Student	Benchmark	Baseline	AutoBuild	Δ
7B	NL2Repo-Bench Pass@1	18.9	41.5	+22.6
7B	Doc2Repo mean pass	29.2	53.2	+24.0
35B-A3B	NL2Repo-Bench Pass@1	22.3	47.3	+25.0
35B-A3B	Doc2Repo mean pass	43.8	61.2	+17.4

5.2 Boundary-replay ablation and controls

We apply the factorial $R_s \in \{0, 1\} \times R_g \in \{0, 1\}$ policy screen to one construction-complete cohort and inject faults into source stripping, candidate cleanup, overlay precedence, test collection, execution, and parsing. Table 4 reports invalid-control escape and valid-task retention. The gates are complementary: R_s mainly catches source/build-side faults and R_g strip/cleanup/overlay/parser faults, so requiring both drives invalid escape to zero while retaining 68% of valid tasks. In the accepted cohort, exact test-ID violations are 0, empty and import-only candidates are rejected at 100%, stub rejection is 96%, mutant kill rate is 90%, adversarial and source-retrieval attacks escape below 2%, and cross-grader agreement is 99.7%.

5.3 Downstream training

We compare an unfiltered baseline arm and a full-strict-policy arm drawn from the same master pool; both satisfy identical upstream feasibility requirements and differ only in whether the exact R_s/R_g , negative-control, and parity policies select examples. Task count, token budget, teacher setting, student model, optimizer, schedule, checkpoint selection, seeds, and evaluation are held fixed, so the contrast estimates the total selection effect. Table 5 shows that AutoBuild-filtered data improves every metric for both students: NL2Repo-Bench Pass@1 rises by 22.6 and 25.0 points for the 7B and 35B-A3B students, and Doc2Repo mean test pass rate by 24.0 and 17.4 points. The gains hold across model scale and both benchmarks, indicating that the recovered label fidelity of Section 4 translates into downstream capability rather than benchmark-specific overfitting.

6 Related Work

Whole-repository generation. NL2Repo-Bench gives an agent one requirements document and an empty workspace, then evaluates the packaged Python library in a separate test image using the upstream pytest suite; it reports mean test pass rate and Pass@1 [3]. Concurrent BeyondSWE evaluates four task families; *Doc2Repo* is its 50-instance document-to-repository subset rather than a separate paper [2]. It reports mean pass rate and the numbers of fully correct and at-least-90%-correct repositories. BeyondSWE already extracts a generated repository from the agent workspace, applies it in a fresh container, restores evaluator tests, and scores it. We therefore do not claim fresh-container judging, hidden-test restoration, or document-to-repository evaluation as new.

Executable task construction. SWE-smith constructs working Python environments before synthesizing faults that break previously passing tests [7]. Concurrent ScaleSWE coordinates environment, test, and test-grounded problem-statement agents and retains tasks with the required buggy/patched

transitions [8]. RepoLaunch produces reproducible builds, per-test commands, and structured status parsers across repositories and platforms [4]. Closest to our data setting, concurrent DeNovoSWE profiles repositories with tests and coverage, synthesizes documentation with critic–repair loops, strips source and recovery channels, filters trajectories, and evaluates downstream transfer to NL2RepoBench and BeyondSWE-Doc2Repo [9]. Environment-first construction, test-grounded specifications, source stripping, and score-based filtering are consequently prior or concurrent ingredients.

Audit and lifecycle validation. SWE-Bench+ audits solution leakage and weak tests in repository-repair tasks [1]. Concurrent Auto Benchmark Audit scales agentic detection of hidden environment dependencies, specification gaps, and brittle graders, then studies ranking changes after expert-validated filtering [6]. Concurrent Change2Task validates healthy, constructed-task, and restored repository states when transferring historical changes to modern revisions [5]. Thus, neither benchmark auditing nor multi-stage executable validation is by itself new.

Table 6: Positioning against the closest systems. “Not reported” means not stated as a primary guarantee in the cited paper. The last row summarizes AutoBuild’s enforced contract.

System	Fresh grader	Transform	Reference lifecycle	IDs / invalid controls
BeyondSWE	yes	extract submission; restore tests	not reported as a construction gate	not reported
DeNovoSWE	yes	strip source and tests	source-side pass/coverage filter	no strict contract reported
Change2Task	task environments	reconstruct history into a task	healthy–task–restored validation	regression, scope, and fidelity checks
AutoBuild	yes	strip source; sanitize candidate	exact source and post-transform replay	exact IDs, negatives, and parity enforced

Scope. Our focus is the disagreement introduced when isolation strips source and tests, sanitizes exports, overlays candidates, collects tests, and aggregates scores. AutoBuild replays a known-good reference through the deployed path and enforces test identities, process status, denominators, negative controls, and grader parity; matched-budget training measures the downstream effect.

7 Limitations and Broader Impacts

Scope. The implementation and experiments cover Python/pytest; other frameworks need their own identity parsers and cleanup policies. Training measures the full strict policy rather than separate R_s and R_g effects, which require four matched arms. Repository-level license manifests are not released.

Guarantees. Reference replay with negative controls is not a semantic oracle: it cannot prove specification completeness, test adequacy, or decontamination. Generator and grader can share mistakes, and a fresh judge can omit a legal dependency; held-out tests plus network, process, filesystem, and secret isolation remain necessary.

Broader impacts. Reliable labels reduce wasted compute and misleading capability claims. Repository harvesting nevertheless raises license, privacy, and supply-chain risks. Releases should preserve provenance and licenses, scan secrets, honor removals, and red-team source leakage and false rejection.

8 Conclusion

Repository-task validity depends on both judge state and delivery transformations. AutoBuild combines clean-room grading with exact reference replay, negative controls, and grader parity. This contract recovers thousands of false-zero labels and improves downstream repository generation by making scores depend on exported artifacts under fixed judge state.

References

- [1] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for LLMs, 2024. URL <https://arxiv.org/abs/2410.06992>.
- [2] Guoxin Chen, Fanzhe Meng, Jiale Zhao, Minghao Li, Daixuan Cheng, Huatong Song, Jie Chen, Yuzhi Lin, Hui Chen, Xin Zhao, Ruihua Song, Chang Liu, Cheng Chen, Kai Jia, and Ji-Rong Wen. BeyondSWE: Can current code agent survive beyond single-repo bug fixing?, 2026. URL <https://arxiv.org/abs/2603.03194>.
- [3] Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, Kai Hua, Juntao Lin, Pai Liu, Mingchen Li, Xuanguang Pan, Zifan Peng, Yujia Qin, Yong Shan, Zhewen Tan, Weihao Xie, Zihan Wang, Yishuo Yuan, Jiayu Zhang, Enduo Zhao, Yunfei Zhao, He Zhu, Liya Zhu, Chenyang Zou, Ming Ding, Jianpeng Jiao, Jiaheng Liu, Minghao Liu, Qian Liu, Chongyang Tao, Jian Yang, Tong Yang, Zhaoxiang Zhang, Xinjie Chen, Wenhao Huang, and Ge Zhang. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents, 2025. URL <https://arxiv.org/abs/2512.12730>.
- [4] Kenan Li, Rongzhi Li, Linghao Zhang, Qirui Jin, Liao Zhu, Xiaosong Huang, Geng Zhang, Yikai Zhang, Shilin He, Chengxing Xie, Xin Zhang, Zijian Jin, Bowen Li, Chaoyun Zhang, Yu Kang, Yufan Huang, Elsie Nallipogu, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. RepoLaunch: Automating build and management of code repositories across languages and platforms, 2026. URL <https://arxiv.org/abs/2603.05026>.
- [5] Haomin Qi, Xingliang Wang, Xuanqi Gao, Baihui Sang, Xin Zhang, Minghua Ma, Pengfei Gao, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Change2Task: From repository changes to executable coding agent tasks and environments, 2026. URL <https://arxiv.org/abs/2607.28591>.
- [6] Junlin Wang, Federico Bianchi, Shang Zhu, Fan Nie, Yongchan Kwon, Bhuwan Dhingra, and James Zou. Automated benchmark auditing for AI agents and large language models, 2026. URL <https://arxiv.org/abs/2605.26079>.
- [7] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/8b86cf5ace600c48fd188efbb8dedec8-Abstract-Datasets_and_Benchmarks_Track.html.
- [8] Jiale Zhao, Guoxin Chen, Fanzhe Meng, Minghao Li, Jie Chen, Hui Xu, Yongshuai Sun, Wayne Xin Zhao, Ruihua Song, Yuan Zhang, Peng Wang, Cheng Chen, Ji-Rong Wen, and Kai Jia. Immersion in the GitHub universe: Scaling coding agents to mastery, 2026. URL <https://arxiv.org/abs/2602.09892>.
- [9] Jiale Zhao, Guoxin Chen, Fanzhe Meng, Wayne Xin Zhao, Ruihua Song, Ji-Rong Wen, and Kai Jia. DeNovoSWE: Scaling long-horizon environments for generating entire repositories from scratch, 2026. URL <https://arxiv.org/abs/2606.10728>.

A Retrospective Re-grading Detail

Re-grading the full production cohort through the AutoBuild grading path changed roughly 10,000 of 35,503 rollout scores, of which more than 4,000 moved from zero to a positive value. The zero-to-positive transitions are the recovered false zeros: valid implementations that the drifted evaluator had penalized for a packaging or cleanup fault. The changed-score count is reported to the nearest thousand and the zero-to-positive count as a strict lower bound from the audit tooling.

35,503	rollouts re-evaluated	EXACT
nearly 10,000	scores corrected	ROUNDED COUNT
more than 4,000	false zeros recovered	STRICT LOWER BOUND

Figure 7: Retrospective re-grading of 35,503 production rollouts: the full cohort, the $\approx 10,000$ scores corrected, and the $> 4,000$ false zeros recovered after closing evaluation-path drift.

B Claims-to-Evidence Ledger

Claim	Evidence
Source-side triage gate filters un-buildable snapshots	build runner blocks completion below a parsed 0.9 threshold before exact replay
Task artifacts share executable evidence	source/test tools, AST inventory, coverage, passing node IDs from one snapshot
Exact boundary replays are batch-enforced	R_s and R_g run per task with test-identity and outcome equality, failing closed
Cleanup defects were prevalent and are now blocked	86/470 and 103-ID cohorts; retrospective re-grading of 35,503 rollouts
Closing drift recovers false-zero labels	$> 4,000$ zero-to-positive transitions on the production cohort
Filtering improves downstream capability	matched-budget 7B/35B-A3B gains on NL2Repo-Bench and Doc2Repo
Negative controls and parity are enforced	empty/stub/mutant/adversarial rejection and cross-grader agreement

C Boundary-Aligned Replay Procedure

```

for each repository snapshot r:
  I_src, C, parser, per_test = build_and_organize(r)
  triage = execute_coarse(I_src, C)
  require triage.total > 0
  require triage.passed / triage.total >= tau

  ids = freeze(per_test.passing_ids)
  require ids is nonempty
  R_src = execute_exact(I_src, C, ids)          # target
  require R_src.return_code == 0
  require R_src.collected_ids == ids
  require R_src.passed_ids == ids
  require R_src.failed == R_src.errors == R_src.skipped == 0

  doc = generate_spec(source=r.source, tests=r.tests,

```

```

329         ast=inventory(r), coverage=profile(r))
330     I_clean, cleanup, manifest = package_clean_task(r, doc)
331
332     R_grade = deployed_judge(candidate=r.source, # target
333                             image=I_clean,
334                             cleanup=cleanup,
335                             tests=manifest)
336     require R_grade.return_code == 0
337     require R_grade.collected_ids == ids
338     require R_grade.passed_ids == ids
339     require R_grade.failed == R_grade.errors == 0
340     require R_grade.skipped == 0
341
342     for negative in [empty, import_stub, behavior_stub,
343                    mutants, adversarial_tests, source_fetch]:
344         require deployed_judge(negative).score < acceptance
345
346     require differential_parity(all_supported_graders)
347     persist immutable inputs, image digests, commands, outputs,
348             parser hash, per-test outcomes, and all gate decisions

```

349 **D Evaluation Details**

350 The primary controlled comparisons are reported in Section 5. Strict two-boundary replay adds
351 roughly $1.7\times$ construction wall-clock over a legacy single build, since each task executes exact source
352 and post-transform replays plus negative controls. Each run retains immutable image and command
353 digests, parser hashes, exact test identities and outcomes, process status, per-control decisions, and
354 cross-grader outcomes for failure adjudication.

355 **E LLM Usage in the Core Method**

356 Language models are core components of environment setup, test-interface organization, and specifi-
357 cation generation. The documentation agent uses repository browsing, test reading, symbol-source
358 retrieval, and runtime-introspection tools. The internal run manifest stores the model identifier,
359 prompt hash, temperature, and seed per artifact; these manifests are not included in the anonymous
360 submission. Writing assistance for this manuscript is separate from the scientific pipeline.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the problem formulation, the implemented protocol, and the validated outcomes; Sections 4 and 5 report the supporting audit, ablation, and downstream results, and Section 7 bounds generalization.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 7 discusses language and framework scope, the limits of reference replay, shared generator/grader failures, held-out testing, and license and release constraints.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper introduces definitions and target invariants, but makes no theorem or proof claim.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: Section 3 and Appendix C specify the protocol, and Section 5 reports the matched design, token budget, schedule, seeds, and evaluation procedure. The internal cohorts and run manifests are not released, so the submission does not provide an external path to fully reproduce every reported result.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The audited implementation and internal rollout data are not presently cleared for anonymous public release; Section 7 states this restriction.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [No]

Justification: Section 5 specifies the matched pool, equal 4.4B-token budget, 7B and 35B-A3B students, cosine schedule with peak $4e-4$, three seeds, fixed final-checkpoint selection, and pinned evaluators. Optimizer type, batch size, warmup, update count, exact checkpoint identifiers, and cohort sizes are not included in the present submission.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The downstream table reports means over three independent seeds with fixed final-checkpoint selection, but the present submission does not include per-seed dispersion or confidence intervals; the validity study likewise reports cohort rates without repository-clustered intervals.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: Section 5 fixes the 4.4B-token budget per arm and Appendix D reports the replay cost multiplier, but GPU type, worker count, memory, and end-to-end wall-clock are not included in the present draft.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This is a software-system audit with no human-subject experiments. The draft avoids exposing private repository locations or credentials and discusses responsible artifact release in Section 7.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 7 discusses both reduced label/compute waste and risks involving license compliance, privacy, supply chains, leakage, and false rejection.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This draft does not release a model or dataset. Proposed release safeguards are listed in Section 7.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: Prior work is cited, but a repository-by-repository license and terms-of-use audit is not available for the internal task cohorts; no cohort assets are released with this draft.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper analyzes an internal prototype and introduces no publicly released dataset, model, or code asset in this submission draft.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

678 Justification: The work does not involve crowdsourcing or human-subject research and
679 therefore requires no IRB approval.

680 Guidelines:

- 681 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
682 with human subjects.
- 683 • Depending on the country in which research is conducted, IRB approval (or equivalent)
684 may be required for any human subjects research. If you obtained IRB approval, you
685 should clearly state this in the paper.
- 686 • We recognize that the procedures for this may vary significantly between institutions
687 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
688 guidelines for their institution.
- 689 • For initial submissions, do not include any information that would break anonymity (if
690 applicable), such as the institution conducting the review.

691 16. Declaration of LLM usage

692 Question: Does the paper describe the usage of LLMs if it is an important, original, or
693 non-standard component of the core methods in this research? Note that if the LLM is used
694 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
695 scientific rigor, or originality of the research, declaration is not required.

696 Answer: [Yes]

697 Justification: Appendix E describes the language-model roles in environment and specifica-
698 tion generation and the model-version, prompt-hash, temperature, and seed records stored
699 per artifact.

700 Guidelines:

- 701 • The answer [N/A] means that the core method development in this research does not
702 involve LLMs as any important, original, or non-standard components.
- 703 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
704 be described.